More practice - Binary trees and BSTs

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	3	2
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Binary trees and BSTs | 6 questions | tap a title to jump to it

E	1. Find right-side view of binary tree	PracHub	2025-11-01
M	2. Implement zigzag level-order traversal	PracHub	2025-08-01
M	3. Employee Hierarchy Add / Delete	1Point3Acres	2026-06-10
M	4. Find mode in a trinary search tree	PracHub	2025-11-16
H	5. Huffman Tree Construction	1Point3Acres	2026-06-01
H	6. Compute distance-sum from every tree node	PracHub	2026-01-22

STUDY PATH: question 99 of 167 (more practice)

Priority score 1.7 | Binary trees and BSTs | asked 353 days ago

EASY | IN APP LC 199 | PracHub | Technical Screen | 1 of 6

Find right-side view of binary tree

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 34.9 | 30 DAYS 0 | PRIORITY 111/228

ABOUT THIS QUESTION

Topic: More practice - Binary trees and BSTs

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #199 Binary Tree Right Side View

LeetCode match: LC 199 Binary Tree Right Side View

Posted on PracHub: 2025-11-01 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-right-side-view-of-binary-tree

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode righ
t; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

APPROACH

BFS taking the last node of each level, or DFS visiting right before left and recording the first node seen at each depth. O(n).

HOW TO THINK ABOUT IT

The right side view is the last node of every level, or equivalently the first node you reach at each depth if you visit right children before left. Either way you need the depth of each node, so carry it in the traversal.

Basis: LeetCode 199, Binary Tree Right Side View.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def right_side_view(root):
    levels = level_order(root)  # list of lists
    return [level[-1] for level in levels]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Stores every node's value when only one per level is needed, O(n) space. A depth-first walk visiting the right child first and recording the first value at each depth needs O(h).

OPTIMAL SOLUTION · DFS visiting right before left, first node at each depth

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Left side view, or both.

Swap the order of the two recursive calls. For both views record the first and last node per depth in one pass.

FOLLOW-UP 2 Avoid recursion.

Use the BFS version and keep only the last node per level (no per-level list), or an explicit stack of (node, depth) pushing left before right so that right is popped first.

FOLLOW-UP 3 Top view or bottom view.

Track a horizontal distance per node (left minus one, right plus one) and keep the first (top) or last (bottom) node seen per distance in BFS order.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find right-side view of binary tree
#
# You are given the root of a binary tree. From the
# **right side** of the tree, at each depth you can see
# exactly one node: the rightmost node at that depth.
# Write a function that returns a list of the values of
# the nodes visible when looking at the tree from the
# right side, ordered from **top to bottom**. --- ###
# Function Signature (one possible form) text
# rightSideView(root: TreeNode) -> List[int] Where
# TreeNode is defined as: text class TreeNode { int val;
# TreeNode left; TreeNode right; } --- ### Example
# Consider this binary tree: text 1 / \ 2 3 \ \ 5 4 The
# right-side view is: text [1, 3, 4] Explain the algorithm
# you would use, including its time and space complexity.
# You may use either breadth-first search (BFS) or depth-
# first search (DFS).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Left side view, or both.
# 2. Avoid recursion.
# 3. Top view or bottom view.
#
# INTERVIEW NOTES
# The right side view is the last node of every level, or
# equivalently the first node you reach at each depth if
# you visit right children before left. Either way you
# need the depth of each node, so carry it in the
# traversal.
#
# BRUTE FORCE (say it first): Collect every level, then
# take the last of each
# Build the full list of nodes per level with a queue and
# return the last value of each list. It stores every node
# of the tree when only one per level is needed. Time
# O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Collect every level, then
# take the last of each | time O(n)
def right_side_levels(root):
    if not root:
```

```
        return []
    levels, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        levels.append(level)
    return [lv[-1] for lv in levels]


# BASE QUESTION (optimal): DFS visiting right before left,
# first node at each depth
# Idea: Visit the right child first. The first node you
# meet at a depth that is not yet recorded is the
# rightmost one at that depth. Only O(height) memory is
# needed.
def right_side_view(root):
    # DFS, right child first: the first node seen at each
    # depth is the rightmost one
    out = []

    def dfs(node, depth):
        if node is None:
```

```
            return
        if depth == len(out):
            out.append(node.val)
        dfs(node.right, depth + 1)
        dfs(node.left, depth + 1)

    dfs(root, 0)
    return out


# FOLLOW-UP 1: Left side view, or both.
# Answer: Swap the order of the two recursive calls. For
# both views record the first and last node per depth in
# one pass.
def left_and_right_views(root):
    # both views in one pass: first and last node per
    # level
```

```
    left, right = [], []
    queue = deque([root] if root else [])
    while queue:
        left.append(queue[0].val)
        right.append(queue[-1].val)
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return left, right


# FOLLOW-UP 2: Avoid recursion.
# Answer: Use the BFS version and keep only the last node
# per level (no per-level list), or an explicit stack of
# (node, depth) pushing left before right so that right is
# popped first.
def right_side_view_bfs(root):
    # no recursion: keep only the LAST node of every level
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append(queue[-1].val)
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return out


# FOLLOW-UP 3: Top view or bottom view.
# Answer: Track a horizontal distance per node (left minus
# one, right plus one) and keep the first (top) or last
# (bottom) node seen per distance in BFS order.
def top_and_bottom_view(root):
    # horizontal distance: left child -1, right child +1
    top, bottom = {}, {}
    queue = deque([(root, 0)] if root else [])
    while queue:
        node, hd = queue.popleft()
        # the first node seen at a distance is the top
        # one;
        # the last one seen is the bottom one
        top.setdefault(hd, node.val)
```

```
        bottom[hd] = node.val
        if node.left:
            queue.append((node.left, hd - 1))
        if node.right:
            queue.append((node.right, hd + 1))
    order = sorted(top)
    return ([top[h] for h in order],
            [bottom[h] for h in order])
```

```
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


if __name__ == "__main__":
    #     1
    #    / \
    #   2   3
    #    \    \
    #     5    4
    tree = TreeNode(1, TreeNode(2, None, TreeNode(5)),
                    TreeNode(3, None, TreeNode(4)))
    assert right_side_view(tree) == [1, 3, 4]
    assert right_side_view_bfs(tree) == [1, 3, 4]
    views = left_and_right_views(tree)
    assert views == ([1, 2, 5], [1, 3, 4])
    assert right_side_view(None) == []
    # a left-only tree is still seen from the right
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))
    assert right_side_view(chain) == [1, 2, 3]
    top, bottom = top_and_bottom_view(tree)
    assert top == [2, 1, 3, 4] and bottom == [2, 5, 3, 4]
```

FUNCTION AND TEST CASES

Signature

```
def rightSideView(level_order: list[int | None]) -> list[i
nt]:
```

Sample call

```
rightSideView([1, 2, 3, None, 5, None, 4])
```

Test cases

Input	Expected
`([1, 2, 3, None, 5, None, 4],)`	`[1, 3, 4]`
`([1, 2, 3, 4],)`	`[1, 3, 4]`

STUDY PATH: question 100 of 167 (more practice)

Priority score 2.55 | Binary trees and BSTs | asked 445 days ago

MEDIUM | IN APP LC 103 | PracHub | Technical Screen | 2 of 6

Implement zigzag level-order traversal

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 38.7 | 30 DAYS 0 | PRIORITY 96/228

ABOUT THIS QUESTION

Topic: More practice - Binary trees and BSTs

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #103 Binary Tree Zigzag Level Order Traversal

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/implement-zigzag-level-order-traversal

QUESTION

Given the root of a binary tree, return the node values in zigzag level order: visit nodes level by level, reading the first level from left to right, the next from right to left, and so on alternating each level. Output a list of lists of integers where each inner list contains the values for one level in the required direction. Aim for O(n) time and O(w) extra space, where n is the number of nodes and w is the maximum width of the tree.

APPROACH

BFS by level; reverse (or use a deque appending left/right) on alternate levels. O(n).

HOW TO THINK ABOUT IT

It is a plain level-order traversal where every second level is read in the other direction. Either collect each level and reverse the odd ones, or fill each level from the correct end of a deque so no reversal is needed.

Basis: LeetCode 103, Binary Tree Zigzag Level Order Traversal.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Same O(n) time, plus an extra pass copying the reversed levels. Filling each level from the left or right end of a deque as it is built needs no reversal.

OPTIMAL SOLUTION · BFS filling each level from the left or the right

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bottom-up level order (leaf level first).

Run the normal traversal and reverse the list of levels at the end.

FOLLOW-UP 2 Do it with DFS.

Pass the depth; append the value to levels[depth], at the end when the depth is even and at the front (insert at 0 or a deque) when odd. Recursion depth is the tree height.

FOLLOW-UP 3 N-ary tree.

Replace the two children by the list of children; the direction flag works the same way.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement zigzag level-order traversal
#
# Given the root of a binary tree, return the node values
# in zigzag level order: visit nodes level by level,
# reading the first level from left to right, the next
# from right to left, and so on alternating each level.
# Output a list of lists of integers where each inner list
# contains the values for one level in the required
# direction. Aim for O(n) time and O(w) extra space, where
# n is the number of nodes and w is the maximum width of
# the tree.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bottom-up level order (leaf level first).
# 2. Do it with DFS.
# 3. N-ary tree.
#
# INTERVIEW NOTES
# It is a plain level-order traversal where every second
# level is read in the other direction. Either collect
# each level and reverse the odd ones, or fill each level
# from the correct end of a deque so no reversal is
# needed.
#
# BRUTE FORCE (say it first): Level order, then reverse
# every second level
```

```python
# Do a normal breadth-first traversal collecting each
# level, and reverse the levels with odd index at the end.
# Easy to get right, with one extra pass over those
# levels. Time O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Level order, then reverse
# every second level | time O(n)
def zigzag_reverse(root):
    levels, queue = [], deque([root] if root else [])
    while queue:
        levels.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(
                c for c in (n.left, n.right) if c
            )
    return [
        lv[::-1] if i % 2 else lv
        for i, lv in enumerate(levels)
    ]


# BASE QUESTION (optimal): BFS filling each level from the
# left or the right
# Idea: Keep a left-to-right flag. For each level build a
# deque; append on the right when going left to right and
# on the left when going right to left. Flip the flag
# after each level.
def zigzag_level_order(root):
    # BFS by levels; flip the direction on every level
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            if ltr:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        ltr = not ltr
    return out


# FOLLOW-UP 1: Bottom-up level order (leaf level first).
# Answer: Run the normal traversal and reverse the list of
# levels at the end.
def level_order_bottom_up(root):
    # leaf level first: the normal traversal, reversed at
    # the end
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return out[::-1]


# FOLLOW-UP 2: Do it with DFS.
# Answer: Pass the depth; append the value to
# levels[depth], at the end when the depth is even and at
# the front (insert at 0 or a deque) when odd. Recursion
# depth is the tree height.
def zigzag_dfs(root):
    # DFS: pass the depth; odd levels are filled at the
    # front
    levels = []

    def go(node, depth):
        if node is None:
            return
        if depth == len(levels):
            levels.append(deque())
        if depth % 2 == 0:
            levels[depth].append(node.val)
        else:
            levels[depth].appendleft(node.val)
        go(node.left, depth + 1)
        go(node.right, depth + 1)

    go(root, 0)
    return [list(x) for x in levels]


# FOLLOW-UP 3: N-ary tree.
# Answer: Replace the two children by the list of
# children; the direction flag works the same way.
def zigzag_nary(root):
    # an n-ary tree: a list of children instead of two
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        vals = [n.val for n in queue]
        out.append(vals if ltr else vals[::-1])
        for _ in range(len(queue)):
            queue.extend(queue.popleft().children)
        ltr = not ltr
    return out



# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


class NNode:
    def __init__(self, val, children=()):
        self.val, self.children = val, list(children)


if __name__ == "__main__":
    right = TreeNode(20, TreeNode(15), TreeNode(7))
    tree = TreeNode(3, TreeNode(9), right)
    want = [[3], [20, 9], [15, 7]]
    assert zigzag_level_order(tree) == want
    assert zigzag_dfs(tree) == want
    up = level_order_bottom_up(tree)
    assert up == [[15, 7], [9, 20], [3]]
    assert zigzag_level_order(None) == []
    assert zigzag_dfs(None) == []
    kids = [NNode(2, [NNode(5)]), NNode(3), NNode(4)]
    root = NNode(1, kids)
    assert zigzag_nary(root) == [[1], [4, 3, 2], [5]]
```

FUNCTION AND TEST CASES

Signature

```
def zigzagLevelOrder(values: list[int | None]) -> list[list[int]]:
```

Sample call

```
zigzagLevelOrder([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
`([3, 9, 20, None, None, 15, 7],)`	`[[3], [20, 9], [15, 7]]`
`([1],)`	`[[1]]`

STUDY PATH: question 101 of 167 (more practice)

Priority score 2.5 | Binary trees and BSTs | asked 132 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 3 of 6

Employee Hierarchy Add / Delete

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 227/228**

ABOUT THIS QUESTION

Topic: More practice - Binary trees and BSTs

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/employee-hierarchy-add-delete

QUESTION

Design an employee hierarchy class that supports adding employees under managers and deleting employees by id. Follow-ups cover deleting managers, deleting the CEO, cycle handling, and choosing a replacement manager by seniority.SWEtreehashmapobject-designmediumFrequencySingle reportLast asked2026-06-10Stageonsite-coding

APPROACH

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what happens when a manager is deleted (promote a report, reassign to the grandparent, or reject), what happens when the CEO is deleted, whether adding can create a cycle, and how seniority is defined.

BRUTE FORCE TO SAY OUT LOUD

Store parent pointers only and search the whole tree to find children on every delete. O(n).

OPTIMAL IDEA

Keep id to node, and for each node a parent pointer and a children set. Add: validate the manager exists and the new id is free. Delete: take the children of the removed node and attach them to a replacement (the deleted node's parent, or a chosen successor), update pointers, and remove it from its parent's children. CEO needs a stated policy. Compare candidates by (level descending, insertion order ascending) for the seniority rule.

DRY-RUN THESE CASES

Delete a leaf, a middle manager, the CEO, an unknown id, adding with a missing manager.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Cycle detection by walking up parent pointers, bulk deletes, or a history of changes.

PITFALLS

Leaving dangling children references. Not choosing a policy for the root.

HOW TO THINK ABOUT IT

A hierarchy is a tree you update, so store links in both directions (parent per node, children per node). Then every operation touches only the neighbours of one node instead of searching the tree. For each delete case (leaf, middle, root) say the policy first, because the data structure is easy once the rule is fixed.

Basis: Written. Assumed policies (confirm them): deleting a manager hands their reports to that manager's own manager; deleting the CEO is allowed only if exactly one report exists (that report becomes CEO), otherwise it is rejected; ids are unique.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
manager = {}  # employee -> manager
def delete(e):
    reports = [x for x in manager
               if manager[x] == e]
    for x in reports:
        manager[x] = manager[e]
    del manager[e]
```

TIME	SPACE
O(n) per delete	**O(n)**

WHY IT IS SLOW

Every delete scans all employees, O(n). Keeping a children set per employee makes it O(number of reports).

OPTIMAL SOLUTION · Parent map plus children sets

TIME	SPACE
O(r) per delete where r is the number of reports	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support moving an employee under another manager and reject cycles.

Walk up from the proposed new manager through manager pointers; if you meet the employee being moved, the move would put a node under its own descendant, so reject it. O(depth).

FOLLOW-UP 2 When a manager leaves, promote the most senior report instead of the grandparent.

Store a hire order (insertion counter) per employee and pick the report with the smallest one: the new manager takes over the other reports. Say how you break ties.

FOLLOW-UP 3 The CEO leaves.

Pick a policy and say it: reject, promote the most senior report (the rest report to them), or leave the company rootless and hold a set of roots. The structure above needs only a promoted root.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Employee Hierarchy Add / Delete
#
# Design an employee hierarchy class that supports adding
# employees under managers and deleting employees by id.
# Follow-ups cover deleting managers, deleting the CEO,
# cycle handling, and choosing a replacement manager by
# seniority.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support moving an employee under another manager and
# reject cycles.
# 2. When a manager leaves, promote the most senior report
# instead of the grandparent.
# 3. The CEO leaves.
#
# INTERVIEW NOTES
# A hierarchy is a tree you update, so store links in both
# directions (parent per node, children per node). Then
# every operation touches only the neighbours of one node
# instead of searching the tree. For each delete case
# (leaf, middle, root) say the policy first, because the
# data structure is easy once the rule is fixed.
#
# BRUTE FORCE (say it first): Store only parent pointers
# and search for children
# Keep a dict from employee to manager. To delete, scan
# all entries to find the reports and reassign them.
# Correct, but every delete is a full scan. Time O(n) per
# delete. Space O(n).


# BRUTE FORCE (say it first): Store only parent pointers
# and search for children | time O(n) per delete
class OrgBrute:
    def __init__(self, ceo):
        self.manager = {ceo: None}

    def add(self, emp, mgr):
        if (
            emp in self.manager
            or mgr not in self.manager
        ):
            return False
        self.manager[emp] = mgr
        return True
```

```
    def delete(self, emp):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        reports = [
            e
            for e, m in self.manager.items()
            if m == emp
        ]
        if mgr is None and len(reports) != 1:
            return False
        for r in reports:
            self.manager[r] = mgr
        del self.manager[emp]
        return True


# BASE QUESTION (optimal): Parent map plus children sets
# Idea: Keep manager[emp] and reports[emp] (a set). Add
# inserts into both. Delete reattaches each report to the
# deleted node's manager and removes the node from its
# manager's set.
# FOLLOW-UP 1: Support moving an employee under another
# manager and reject cycles.
# Answer: Walk up from the proposed new manager through
# manager pointers; if you meet the employee being moved,
# the move would put a node under its own descendant, so
# reject it. O(depth).
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        # hire order, used for "most senior" choices
        self.hired = {ceo: 0}
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
```

```
        self.clock += 1
        return True

    def senior(self, people):
        # the person hired earliest
        return min(people, key=lambda p: self.hired[p])

    def delete(self, emp, policy="grandparent"):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        kids = set(self.reports[emp])
        # FOLLOW-UP 3: the CEO leaves
        if mgr is None and not kids:
            # the CEO alone: nothing could take over
            return False
        # FOLLOW-UP 2: promote the most senior report
        if policy == "senior" or mgr is None:
            # the most senior report takes the place of
            # emp
            heir = self.senior(kids) if kids else None
        else:
            heir = mgr
        for r in kids:
            if r != heir:
                self.reassign(r, heir)
        if mgr is None:
            # the promoted report becomes the new CEO
            self.root = heir
```

```
            self.manager[heir] = None
        elif policy == "senior" and heir != mgr:
            self.reassign(heir, mgr)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True

    def reassign(self, emp, new_mgr):
        old = self.manager[emp]
        if old is not None and old in self.reports:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)

    # FOLLOW-UP 1: move an employee, reject cycles
```

```
    def move(self, emp, new_mgr):
        # walk up from the new manager: meeting emp is a
        # cycle
        node = new_mgr
        while node is not None:
            if node == emp:
                return False
            node = self.manager[node]
        self.reassign(emp, new_mgr)
        return True


if __name__ == "__main__":
    o = Org("ceo")
    for e, m in [("a", "ceo"), ("b", "a"), ("c", "a")]:
        assert o.add(e, m)
    assert not o.add("a", "ceo")
    assert not o.add("x", "nobody")
    # deleting a manager: reports go to the grandparent
    assert o.delete("a")
    assert o.manager["b"] == "ceo"
    assert o.manager["c"] == "ceo"
    # a move that would create a cycle is rejected
    o.add("d", "b")
    assert not o.move("b", "d") and o.move("d", "c")
    # promote the most senior report instead
    p = Org("ceo")
    for e, m in [("m", "ceo"), ("x", "m"), ("y", "m")]:
        p.add(e, m)
    assert p.delete("m", "senior")
    assert p.manager["x"] == "ceo"
    assert p.manager["y"] == "x"
    # the CEO leaves: the most senior report takes over
    q = Org("ceo")
    q.add("a", "ceo")
    q.add("b", "ceo")
    assert q.delete("ceo")
    assert q.root == "a" and q.manager["b"] == "a"
    assert not Org("solo").delete("solo")
```

NOTES FROM CANDIDATES

Use maps from id to node and id to children; keep parent pointers so deletes do not require a full traversal.
CEO deletion needs an explicit policy: reject, promote a child, or create a new root. State the policy and keep it consistent.
For the seniority follow-up, maintain insertion order on nodes and compare (level desc, insertion_order asc).

PREPARATION

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

STUDY PATH: question 102 of 167 (more practice)

Priority score 2.02 | Binary trees and BSTs | asked 338 days ago

MEDIUM | IN APP LC 501 | PracHub | Technical Screen | 4 of 6

Find mode in a trinary search tree

POSTED 2025-11-16 | 338 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 137/228

ABOUT THIS QUESTION

Topic: More practice - Binary trees and BSTs

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #501 Find Mode in Binary Search Tree

LeetCode match: LC 501 Find Mode in Binary Search Tree

Posted on PracHub: 2025-11-16 (updated 2026-06-20)

Original:
https://prachub.com/coding-questions/find-mode-in-a-trinary-search-tree

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val N
ode middle; // all values = val Node right; // all val
ues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

QUESTION

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

Hints

In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.
Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer
Examples
Input: ([], [], [], [], -1)
Expected Output: []

QUESTION

Explanation: The tree is empty, so there are no modes.
Input: ([-2147483648], [-1], [-1], [-1], 0)
Expected Output: [-2147483648]
Explanation: A single-node tree has that node's value as the only mode.
Hints
Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

APPROACH

In-order traversal (left, middle, right) so equal values are consecutive; track current run length and the max; O(h) extra space.

HOW TO THINK ABOUT IT

In a valid search tree an in-order walk (left, middle, right) visits values in sorted order, so equal values are adjacent and you can count runs with a single running counter and O(height) memory instead of a dictionary. If the tree may be broken, the order guarantee is gone and you need a real frequency table.

Basis: LeetCode 501, Find Mode in Binary Search Tree, with a middle child for equal values. Valid case: left < val, middle = val, right > val.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def modes(root):
    counts = Counter()
    stack = [root]
    while stack:
        node = stack.pop()
        if node:
            counts[node.val] += 1
            stack += [node.left, node.middle,
                      node.right]
    top = max(counts.values())
    return [v for v in counts
            if counts[v] == top]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but memory for every distinct value. In a valid tree an in-order walk (left, middle, right) gives equal values consecutively, so a running counter needs only O(height) space.

OPTIMAL SOLUTION · In-order traversal counting runs (O(height) extra space)

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree may be broken (it does not respect the ordering).

In-order no longer gives sorted values, so equal values need not be adjacent. Fall back to a frequency table over a full traversal (the Counter version): O(n) time, O(distinct values) space.

FOLLOW-UP 2 Only O(1) extra space (no stack, no dictionary).

Use Morris traversal for the in-order walk (temporary threads) with the same running-count logic; it mutates the tree briefly, so restore the pointers. Adapting it to the middle child needs care.

FOLLOW-UP 3 The tree is huge and values arrive in a stream.

For a stream use a count dictionary (or a Misra-Gries summary if only frequent values matter); the sorted-order trick needs the whole tree in order. **Cost:** O(1) per value with a counter and O(d) memory for the distinct values (or O(1 / epsilon) with Misra-Gries).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find mode in a trinary search tree
#
# You are given the root of a **trinary search tree**
# where each node has up to three children: text class
# Node { int val; Node left; // all values < val Node
# middle; // all values = val Node right; // all values >
# val } Assume the tree is intended to satisfy the
# ordering rules above. ## Task Return the **mode(s)** of
# the tree: the value(s) that occur most frequently among
# all nodes. - If there is one mode, return a single-
# element list. - If multiple values tie for highest
# frequency, return **all** such values (in any order). ##
# Constraints (typical) - Number of nodes n can be large
# (e.g., up to 10^5). - Values can be any 32-bit signed
# integers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree may be broken (it does not respect the
# ordering).
# 2. Only O(1) extra space (no stack, no dictionary).
# 3. The tree is huge and values arrive in a stream.
#
# INTERVIEW NOTES
# In a valid search tree an in-order walk (left, middle,
# right) visits values in sorted order, so equal values
```

```
# are adjacent and you can count runs with a single
# running counter and O(height) memory instead of a
# dictionary. If the tree may be broken, the order
# guarantee is gone and you need a real frequency table.
#
# BRUTE FORCE (say it first): Collect every value in a
# Counter
# Traverse the whole tree, count each value and return the
# ones with the highest count. Correct for any tree (even
# a broken one) and the oracle; it uses memory
# proportional to the number of distinct values. Time
# O(n). Space O(n).


from collections import Counter


# BRUTE FORCE (say it first): Collect every value in a
# Counter | time O(n)
def modes_counter(root):
```

```
    c, stack = Counter(), [root] if root else []
    while stack:
        x = stack.pop()
        c[x.val] += 1
        stack.extend(
            ch
            for ch in (x.left, x.middle, x.right)
            if ch
        )
    if not c:
        return []
    top = max(c.values())
    return sorted(v for v, k in c.items() if k == top)


# BASE QUESTION (optimal): In-order traversal counting
# runs (O(height) extra space)
# Idea: Walk left, middle, right with an explicit stack.
def find_modes(root):
    # in-order: equal values are next to each other, so a
    # running count needs no dictionary
    best, modes, cur, run = 0, [], None, 0

    def visit(v):
        nonlocal best, modes, cur, run
        run = run + 1 if v == cur else 1
        cur = v
        if run > best:
            best, modes = run, [v]
        elif run == best and v not in modes:
            modes.append(v)

    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        visit(node.val)
        # the middle chain repeats node.val once per child
        mid = node.middle
        while mid:
            visit(mid.val)
            mid = mid.middle
        node = node.right
```

```
    return sorted(modes)


# FOLLOW-UP 1: The tree may be broken (it does not respect
# the ordering).
# Answer: In-order no longer gives sorted values, so equal
# values need not be adjacent. Fall back to a frequency
# table over a full traversal (the Counter version): O(n)
# time, O(distinct values) space.
def find_modes_counter(root):
    # a broken tree (ordering not respected): count
    # everything
    counts, stack = Counter(), [root] if root else []
    while stack:
        node = stack.pop()
        counts[node.val] += 1
        for child in (node.left, node.middle, node.right):
            if child:
                stack.append(child)
    if not counts:
        return []
    top = max(counts.values())
    return sorted(v for v in counts if counts[v] == top)


# HELPERS for the checks below
class TNode:
    def __init__(self, val, left=None, middle=None,
                 right=None):
        # left < val, middle == val, right > val
        self.val, self.left = val, left
        self.middle, self.right = middle, right


if __name__ == "__main__":
    #        5
    #     /  |  \
    #    3   5   8
    #   /|       |
    #  3 3       8
    threes = TNode(3, None, TNode(3, None, TNode(3)))
    tree = TNode(5, threes, TNode(5),
                 TNode(8, None, TNode(8)))
    assert find_modes(tree) == [3]
    assert find_modes_counter(tree) == [3]
```

```
    # a tie between 1 and 2
    tie = TNode(2, TNode(1), TNode(2), None)
    assert find_modes(tie) == [2]
    assert find_modes_counter(tie) == [2]
    assert find_modes(None) == []
    assert find_modes_counter(None) == []
    tie2 = TNode(2, TNode(1), None, TNode(3))
    assert find_modes(tie2) == [1, 2, 3]
```

FUNCTION AND TEST CASES

Signature

```
def solution(values: list[int], left: list[int], middle: l
ist[int], right: list[int], root: int) -> list[int]:
```

Sample call

```
solution([], [], [], [], -1)
```

Test cases

Input	Expected
([], [], [], [], -1)	[]
([42], [-1], [-1], [-1], 0)	[42]

STUDY PATH: question 103 of 167 (more practice)

Priority score 2.5 | Binary trees and BSTs | asked 141 days ago | frequency Low

HARD | 1Point3Acres | onsite-coding | 5 of 6

Huffman Tree Construction

ASKED 2026-06-01 | 141 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 189/228**

ABOUT THIS QUESTION

Topic: More practice - Binary trees and BSTs

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/huffman-tree-construction

QUESTION

EXAMPLES

```
{"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45}
    -> standard Huffman tree with f near the root.
```

APPROACH

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether to return the tree, a code table, or the encoded message, how ties are broken, and what to do with one symbol.

BRUTE FORCE TO SAY OUT LOUD

Repeatedly scan for the two smallest weights. O(n^2).

OPTIMAL IDEA

Min-heap of nodes keyed by (weight, counter). Pop two, merge under a new node with the summed weight, push back, until one node remains. Walk the tree assigning '0' to left and '1' to right. O(n log n). Decode by walking the tree bit by bit. The counter prevents comparing nodes on ties.

DRY-RUN THESE CASES

The CLRS example (total cost 224, 'f' gets a one-bit code). One symbol (code '0'). Equal weights.

LIKELY FOLLOW-UPS

Canonical codes, decoding, or the two-queue O(n) method on sorted input.

PITFALLS

Comparing tree nodes on equal weights. Forgetting the single symbol case.

HOW TO THINK ABOUT IT

The two rarest symbols belong at the deepest level as siblings, so merge them first, treat the merged node as a new symbol with the combined weight, and repeat. 'Take the two smallest' is the job of a min-heap; tie-breaking with a counter keeps the heap from comparing tree nodes.

Basis: Classic greedy algorithm (CLRS 16.3); related to LeetCode 1167, Minimum Cost to Connect Sticks.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]  # two smallest
        items = items[2:] + [a + b]
        cost += a + b
    return cost
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Finding the two smallest by scanning costs O(n) per merge and n - 1 merges, O(n^2). A min-heap does each merge in O(log n).

OPTIMAL SOLUTION · Min-heap merge of the two lightest nodes

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the greedy choice optimal?

Exchange argument: in some optimal tree the two least frequent symbols are siblings at the deepest level (swapping a shallower, more frequent symbol with a deeper, less frequent one never increases cost). Merging them into one symbol reduces the problem to n - 1 symbols with the same optimal-substructure property, so induction closes it.

FOLLOW-UP 2 How do you make the code canonical, and why would you?

Keep only each symbol's code length from the tree, sort symbols by (length, symbol), and assign consecutive binary codes. A decoder then needs just the lengths rather than the whole tree, which is how DEFLATE stores its tables.

FOLLOW-UP 3 Time complexity if symbols arrive already sorted?

Two-queue method: one queue holds the sorted leaves, the other the merged nodes (which are produced in nondecreasing order). Always pop the smaller front of the two queues. That is O(n) after sorting.

FOLLOW-UP 4 Make the codes canonical so only the code lengths need to be stored.

Sort symbols by (code length, symbol) and assign consecutive binary numbers, shifting left when the length grows. A decoder rebuilds the codes from the lengths alone (this is how DEFLATE stores its tables).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Huffman Tree Construction
#
# Examples: {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16,
# "f": 45} -> standard Huffman tree with f near the root.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the greedy choice optimal?
# 2. How do you make the code canonical, and why would
# you?
# 3. Time complexity if symbols arrive already sorted?
# 4. Make the codes canonical so only the code lengths
# need to be stored.
#
# INTERVIEW NOTES
# The two rarest symbols belong at the deepest level as
# siblings, so merge them first, treat the merged node as
# a new symbol with the combined weight, and repeat. 'Take
# the two smallest' is the job of a min-heap; tie-breaking
# with a counter keeps the heap from comparing tree nodes.
#
# BRUTE FORCE (say it first): Scan for the two smallest
# weights on every merge
# Keep a list of (weight, tree) items; each round scan for
# the two smallest, merge, and put the result back. The
# same greedy as the heap version, with a linear scan
# instead of a heap operation. Returns the total cost (sum
# of weight * code length). Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Scan for the two smallest
# weights on every merge | time O(n^2)
def huffman_cost_scan(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]
        cost += a + b
        items = items[2:] + [a + b]
    return cost


# HELPERS for the base question
def huffman_lengths(freq):
    # code length of every symbol (depth in the tree)
    if not freq:
        return {}
    if len(freq) == 1:
        # a lone symbol still needs one bit
        return {next(iter(freq)): 1}
    # heap entries: (weight, tie-break, symbols below)
    heap = [(w, i, [s]) for i, (s, w) in
            enumerate(sorted(freq.items()))]
    heapq.heapify(heap)
    depth = {s: 0 for s in freq}
    tie = len(heap)
    while len(heap) > 1:
        # merge the two lightest; everything below goes
        # deeper
        w1, _, a = heapq.heappop(heap)
        w2, _, b = heapq.heappop(heap)
        for s in a + b:
            depth[s] += 1
        heapq.heappush(heap, (w1 + w2, tie, a + b))
        tie += 1
    return depth


# FOLLOW-UP 2: How do you make the code canonical, and why
# would you?
# Answer: Keep only each symbol's code length from the
# tree, sort symbols by (length, symbol), and assign
# consecutive binary codes. A decoder then needs just the
# lengths rather than the whole tree, which is how DEFLATE
# stores its tables.
# FOLLOW-UP 4: Make the codes canonical so only the code
# lengths need to be stored.
# Answer: Sort symbols by (code length, symbol) and assign
# consecutive binary numbers, shifting left when the
# length grows. A decoder rebuilds the codes from the
# lengths alone (this is how DEFLATE stores its tables).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def canonical_codes(lengths):
    # assign consecutive binary numbers by (length,
    # symbol)
    code, prev, out = 0, 0, {}
    for sym, length in sorted(lengths.items(),
                              key=lambda x: (x[1], x[0])):
        # moving to a longer length shifts the code left
        code <<= length - prev
        out[sym] = format(code, f"0{length}b")
        code += 1
        prev = length
    return out


# BASE QUESTION (optimal): Min-heap merge of the two
# lightest nodes
# Idea: Put every symbol in a min-heap keyed by frequency,
# repeatedly pop the two lightest nodes, merge them under
# a new internal node with the summed weight and push it
# back. The last node is the root.
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))


# FOLLOW-UP 3: Time complexity if symbols arrive already
# sorted?
# Answer: Two-queue method: one queue holds the sorted
# leaves, the other the merged nodes (which are produced
# in nondecreasing order). Always pop the smaller front of
# the two queues. That is O(n) after sorting.
def lengths_two_queues(sorted_items):
    # input already sorted by weight: O(n) with two queues
```

```
    from collections import deque
    leaves = deque((w, [s]) for s, w in sorted_items)
    merged = deque()
    depth = {s: 0 for s, _ in sorted_items}

    def pop_smaller():
        # the smaller front of the two queues
        if not merged or (leaves and
                          leaves[0][0] <= merged[0][0]):
            return leaves.popleft()
        return merged.popleft()

    while len(leaves) + len(merged) > 1:
        w1, a = pop_smaller()
        w2, b = pop_smaller()
        for s in a + b:
            depth[s] += 1
        merged.append((w1 + w2, a + b))
    return depth


# HELPERS for the checks below
def decode(bits, codes):
    # read bits until they match a code (prefix-free)
    back = {v: k for k, v in codes.items()}
    out, cur = [], ""
    for b in bits:
        cur += b
        if cur in back:
            out.append(back[cur])
            cur = ""
    return "".join(out)


if __name__ == "__main__":
    freq = {"a": 5, "b": 9, "c": 12,
            "d": 13, "e": 16, "f": 45}
    lengths = huffman_lengths(freq)
    # the most frequent symbol is next to the root
    assert lengths["f"] == 1
    # total cost of the optimal tree
    assert sum(freq[s] * lengths[s] for s in freq) == 224
    codes = huffman_codes(freq)
    # prefix-free: no code starts another code
    for x in codes.values():
        for y in codes.values():
            assert x == y or not y.startswith(x)
    text = "abfadc"
    bits = "".join(codes[ch] for ch in text)
    assert decode(bits, codes) == text
    items = sorted(freq.items(), key=lambda x: x[1])
    two = lengths_two_queues(items)
    assert sum(freq[s] * two[s] for s in freq) == 224
    assert huffman_codes({"x": 3}) == {"x": "0"}
    assert huffman_codes({}) == {}
```

NOTES FROM CANDIDATES

L6 onsite — interviewer expects clean code with a documented tree node, plus a discussion of encoding / decoding correctness.
Common follow-up: how to produce the code table (DFS from root, accumulating bits).
Trade-off discussion: Huffman is optimal for known per-symbol probability but not for adaptive / streaming inputs.

PREPARATION

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

STUDY PATH: question 104 of 167 (more practice)

Priority score 2.22 | Binary trees and BSTs | asked 271 days ago

HARD | IN APP LC 834 | PracHub | Technical Screen | 6 of 6

Compute distance-sum from every tree node

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 123/228

ABOUT THIS QUESTION

Topic: More practice - Binary trees and BSTs

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #834 Sum of Distances in Tree

LeetCode match: LC 834 Sum of Distances in Tree

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/compute-distance-sum-from-every-tree-node

QUESTION

Overview: This question evaluates understanding of tree and graph algorithms, including computation of pairwise distances and techniques for aggregating path lengths across nodes.

APPROACH

Rerooting DP: one DFS computes subtree sizes and the root's distance sum, a second DFS derives each child as ans[child] = ans[parent] - size[child] + (n - size[child]). O(n).

HOW TO THINK ABOUT IT

Computing the sum of distances from each node separately costs O(n) per node. Instead compute it once for the root, then derive the neighbour's answer in O(1): moving the root from a parent to its child makes every node in the child's subtree one step closer and every other node one step farther. That needs only the subtree sizes.

Basis: LeetCode 834, Sum of Distances in Tree (re-rooting dynamic programming).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = []
    for node in range(n):
        dist = bfs(adj, node)
        ans.append(sum(dist))
    return ans
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

BRUTE FORCE

n searches of O(n) each, O(n^2). Compute the answer for the root, then move the root edge by edge using subtree sizes (re-rooting), O(n).

OPTIMAL SOLUTION · Re-rooting with subtree sizes

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Find the node with the smallest distance sum (the tree's centroid).

The answer array above gives it directly: argmin of ans. Equivalent: the centroid, found by walking toward the heavier subtree.

FOLLOW-UP 2 The tree has weighted edges.

Re-rooting still works, but the change when moving the root across an edge of weight w is w * (n - 2 * size[c]) instead of n - 2 * size[c]. Sizes and the first pass are the same.

FOLLOW-UP 3 Maximum distance (eccentricity) from each node instead of the sum.

The farthest node from any node is one end of a diameter; compute distances from both diameter ends and take the maximum per node. O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute distance-sum from every tree node
#
# This question evaluates understanding of tree and graph
# algorithms, including computation of pairwise distances
# and techniques for aggregating path lengths across
# nodes.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Find the node with the smallest distance sum (the
# tree's centroid).
# 2. The tree has weighted edges.
# 3. Maximum distance (eccentricity) from each node
# instead of the sum.
#
# INTERVIEW NOTES
# Computing the sum of distances from each node separately
# costs O(n) per node. Instead compute it once for the
# root, then derive the neighbour's answer in O(1): moving
# the root from a parent to its child makes every node in
# the child's subtree one step closer and every other node
# one step farther. That needs only the subtree sizes.
#
```

```python
# BRUTE FORCE (say it first): BFS from every node
# For each node run a BFS and add up the distances.
# Correct and the oracle; n searches of O(n) each. Time
# O(n^2). Space O(n).


from collections import defaultdict
from collections import deque


# BRUTE FORCE (say it first): BFS from every node | time
# O(n^2)
def distance_sums_brute(n, edges):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    out = []
    for s in range(n):
        dist, queue = {s: 0}, deque([s])
        while queue:
            u = queue.popleft()
            for v in adj[u]:
```

```python
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        out.append(sum(dist.values()))
    return out


# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for e in edges:
        u, v = e[0], e[1]
        w = e[2] if len(e) > 2 else 1
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


def distances_from(adj, s):
    # distance from s to every node (a tree: one pass)
    dist, stack = {s: 0}, [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist


# BASE QUESTION (optimal): Re-rooting with subtree sizes
# Idea: Root the tree at 0 and compute size[v] (subtree
# sizes) and ans[0] = the sum of depths.
def distance_sums(n, edges):
    # sum of distances from every node; edges may be
    # weighted
    adj = build(n, edges)
    parent, order, up = [-1] * n, [0], [0] * n
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v], up[v] = u, w
                order.append(v)
    size = [1] * n
    for u in reversed(order):
```

```python
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    # the root's sum is a plain sum of distances
    ans = [0] * n
    ans[0] = sum(distances_from(adj, 0).values())
    # root moves across an edge of weight w: size[u] nodes
    # get
    # w closer, the other n - size[u] get w farther
    for u in order[1:]:
        shift = up[u] * (n - 2 * size[u])
        ans[u] = ans[parent[u]] + shift
    return ans


# FOLLOW-UP 1: Find the node with the smallest distance
# sum (the tree's centroid).
# Answer: The answer array above gives it directly: argmin
# of ans. Equivalent: the centroid, found by walking
# toward the heavier subtree.
def best_center(n, edges):
    # the node with the smallest distance sum
    sums = distance_sums(n, edges)
    return sums.index(min(sums))


# FOLLOW-UP 3: Maximum distance (eccentricity) from each
# node instead of the sum.
# Answer: The farthest node from any node is one end of a
# diameter; compute distances from both diameter ends and
# take the maximum per node. O(n).
def eccentricities(n, edges):
    # the farthest node from anywhere is an end of a
    # diameter
    adj = build(n, edges)
    d0 = distances_from(adj, 0)
    a = max(d0, key=d0.get)
    da = distances_from(adj, a)
    b = max(da, key=da.get)
    db = distances_from(adj, b)
    return [max(da[v], db[v]) for v in range(n)]


if __name__ == "__main__":
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    want = [8, 12, 6, 10, 10, 10]
```

```python
    assert distance_sums(6, edges) == want
    assert best_center(6, edges) == 2
    assert distance_sums(1, []) == [0]
    # weighted: the path 0 -(2)- 1 -(3)- 2
    w = [[0, 1, 2], [1, 2, 3]]
    assert distance_sums(3, w) == [7, 5, 8]
    # eccentricity: the farthest node from each node
    assert eccentricities(6, edges) == [2, 3, 2, 3, 3, 3]
```

FUNCTION AND TEST CASES

Signature

```python
def solution(n: int, edges: list[list[int]]) -> list[int]:
```

Sample call

```python
solution(1, [])
```

Test cases

Input	Expected
`(1, [])`	`[0]`
`(4, [[0, 1], [1, 2], [1, 3]])`	`[5, 3, 5, 5]`

ALSO REPORTED · 1 more

Tree Distance Sum (Re-root DP) | 1Point3Acres | onsite-coding | asked 2026-01-19 | Single report

https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

The interviewer explicitly liked the "baseline first, optimize later" structure — name the $O(n^2)$ and move on within the first ~10 minutes.

Two-pass technique to memorize:

Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree). Recurrence: subSum[u] += subSum[v] + subSize[v].

Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].

Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

Preparation: Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.

Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.

Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).